PETTYFLOW

Enterprise Petty Cash System & Distributed Financial Ledger Platform

Document Type:	Product Planning & Systems Architecture Specification
Engineering Standard:	Jeff Dean High-Performance Systems Paradigm
Author / Team:	Core Financial Infrastructure Group
Ledger Invariant:	Cryptographic HMAC-SHA256 Immutable Double-Entry
Latency Goal:	Sub-2ms Write Commitment (p99)
Target Throughput:	100,000 TPS Distributed Peak Capacity
Version:	1.0.0-RELEASE

EXECUTIVE ARCHITECTURAL SUMMARY

PettyFlow is designed to solve systemic enterprise petty cash leakage, receipt fraud, and slow float replenishment cycles. Built upon Jeff Dean systems principles, PettyFlow guarantees strict double-entry balance invariants (Σ Debits $\equiv \Sigma$ Credits), tamper-evident SHA-256 cryptographic chain audits, zero-trust multi-tenancy, and sub-second AI receipt OCR extraction. This document establishes the product requirements, domain models, database DDLs, gRPC/REST APIs, and infrastructure specs.

1. System Latency & Performance Budgets

To achieve enterprise-scale throughput and zero loss of financial auditability, PettyFlow defines rigid latency boundaries modelled after Jeff Dean's 'Numbers Every Engineer Should Know'. All system components must fit within these explicit budgets:

Operation	Latency Target (p50)	Latency Target (p99)	Architecture Mechanism
L1/L2 Account State Lookup	50 ns	150 ns	CPU L1/L2 cache ring buffer
In-Memory Balance Verification	15 µs	50 µs	Lock-free memory map
Double-Entry Ledger Commit	600 µs	1.8 ms	Append-only WAL + HMAC
Redis Cache Balance Invalidation	200 µs	800 µs	Async pipeline LUA script
AI OCR Receipt Extraction	850 ms	1.4 s	TensorRT / VLM async worker
Perceptual Fraud Screening	12 ms	35 ms	dHash bit vector comparison
Cross-Region Replica Sync	18 ms	45 ms	Postgres streaming replication

2. Double-Entry Cryptographic Ledger Engine

PettyFlow models all financial activity using strict double-entry accounting. Money cannot be created or destroyed, only moved between accounts. Every posting consists of matched debit and credit legs:

Fundamental Ledger Invariants:

- **Zero-Sum Equation:** $\sum$ Debits - $\sum$ Credits = 0 for every transaction UUID.
- **Fixed-Point Arithmetic:** Amounts stored as 64-bit integers scaled by 10,000 (0.0001 precision).
- **Cryptographic Hash Chain:** Entry H_n = HMAC-SHA256(H_{n-1} || EntryPayload || Timestamp, Key_{Tenant}).

Account Classifications & Balance Invariants

Account Category	Normal Balance	Debit Impact (+)	Credit Impact (-)	PettyFlow Example
ASSET	Debit	Increases Balance	Decreases Balance	Custodial Physical Cash Float
EXPENSE	Debit	Increases Expense	Decreases Expense	Office Supplies / Fuel Receipt
LIABILITY	Credit	Decreases Liability	Increases Liability	Pending Employee Reimbursement
EQUITY	Credit	Decreases Capital	Increases Capital	Corporate Float Capital Allocation

3. Relational & TimescaleDB Database Schema

PettyFlow uses PostgreSQL for transactional metadata and partitioned TimescaleDB tables for audit logging. The schema below enforces multi-tenant isolation via compound foreign keys and immutability triggers:

```
-- Core Multi-Tenant Double-Entry Postings Table
CREATE TABLE pettyflow_postings (
    posting_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id      UUID NOT NULL REFERENCES pettyflow_tenants(tenant_id),
    transaction_id UUID NOT NULL,
    account_id     UUID NOT NULL REFERENCES pettyflow_accounts(account_id),
    amount_scaled  BIGINT NOT NULL, -- Scaled by 10,000 (e.g. $12.50 -> 125000)
    entry_type     VARCHAR(6) NOT NULL CHECK (entry_type IN ('DEBIT', 'CREDIT')),
    previous_hash  BYTEA NOT NULL,
    current_hash   BYTEA NOT NULL,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp()
) PARTITION BY RANGE (created_at);

-- Immutability Guard: Prevent UPDATE or DELETE on posting records
CREATE RULE prevent_postings_alter AS ON UPDATE TO pettyflow_postings DO INSTEAD NOTHING;
CREATE RULE prevent_postings_delete AS ON DELETE TO pettyflow_postings DO INSTEAD NOTHING;
```

4. High-Throughput API Specifications

PettyFlow exposes gRPC endpoints for inter-service communication and OpenAPI REST endpoints for mobile & web apps.

Core Protobuf Service Definition (`float_service.proto`):

```
syntax = "proto3";
package pettyflow.v1;

service FloatService {
  rpc CreateFloatAllocation (CreateFloatRequest) returns (CreateFloatResponse);
  rpc SubmitDisbursement (DisbursementRequest) returns (DisbursementResponse);
  rpc VerifyLedgerIntegrity (IntegrityCheckRequest) returns (IntegrityCheckResponse);
}

message DisbursementRequest {
  string tenant_id = 1;
  string fund_id = 2;
  string custodian_id = 3;
  int64 amount_scaled = 4;
  string currency = 5;
  bytes receipt_hash = 6;
}
```

5. AI OCR & Machine Learning Fraud Detection

To mitigate cash leakage and receipt forgery, PettyFlow deploys a dual-stage AI verification pipeline:

Pipeline Stage	Model / Tech Stack	Function & Invariant
1. Image Preprocessing	OpenCV / CUDA	Deskew, contrast normalisation, noise reduction
2. Document OCR	TrOCR / Vision LLM	Extract merchant, date, tax ID, items, subtotal, tax, total
3. Math Validation	Python SymPy / Deterministic	Verify Subtotal + Tax + Tip ≡ Total
4. Perceptual Dup Check	dHash / Hamming Distance	Flag receipt photos matching prior submissions (≤ 5 bits)
5. Split Tx Detection	Sliding Window Aggregator	Detect multiple disbursements designed to bypass approval limits

6. Zero-Trust Security & Compliance Framework

PettyFlow enforces SOC2 Type II, PCI-DSS v4.0, and GDPR compliance throughout the architecture:

- **Envelope Encryption:** Data at rest encrypted with AES-256-GCM via AWS KMS / HashiCorp Vault. Master key rotated every 90 days.
- **Tenant Context Scoping:** Strict multi-tenancy enforced at ORM level via non-bypassable session variables (`SET LOCAL app.current_tenant`).
- **Immutable WORM Logs:** Audit logs streamed to AWS S3 Object Lock in Compliance Mode (Write Once, Read Many).

7. 12-Week AI Deliverable Roadmap Summary

The following weekly milestones govern the AI code generation and deployment schedule:

Week	Deliverable Focus	Key Engineering Artifacts
Week 1	Core Double-Entry Ledger	Domain ledger, HMAC chain, invariant unit tests
Week 2	Database & Cache Tier	PostgreSQL partitioning, Redis LUA balance cache
Week 3	Float Allocation APIs	gRPC protobuf services, FastAPI REST wrapper
Week 4	Approval Workflow Engine	Deterministic state machine, rule evaluator
Week 5	AI OCR Receipt Ingestion	TrOCR parser pipeline, image preprocessor
Week 6	ML Fraud Screening	Perceptual hash, split transaction detector
Week 7	Card & Wallet Adapters	Stripe Virtual Cards, Mobile Money API
Week 8	ERP & Bank Connectors	SAP S/4HANA, NetSuite, ISO 20022 bank feed
Week 9	Reconciliation Engine	3-Way matching, daily variance closing
Week 10	Zero-Trust & KMS Security	Envelope encryption, SOC2 WORM audit log
Week 11	Analytics & Multi-Currency	ECB rate sync, real-time analytics dashboard
Week 12	100k TPS Load & Launch	Locust stress tests, Kubernetes Helm deploy